

Práctica 03

Asignatura: Tecnología de Objetos

NOTA: _____

Apellidos y Nombres: Bedregal Coaguila, Karla Miluska

Indicaciones:

- La nota será ponderada en la evaluación CONTINUA
- Responda de manera clara
- Tiempo de duración del examen: 50 minutos

Ejercicios de programación en C++

Conceptos base

Clase abstracta. Clase que sirve como molde y de la que no se pueden crear objetos. En C++ se vuelve abstracta cuando tiene al menos un método virtual puro.

Método virtual puro. Método declarado con `= 0` y sin cuerpo en la clase base. Obliga a cada clase hija a implementarlo; si no lo hace, la hija también queda abstracta.

Herencia. Las clases derivadas se crean con `class Hija : public Padre` y reciben los atributos y métodos del padre.

override. Indica que se sobrescribe un método virtual del padre. Si la firma no coincide, el compilador marca error.

Polimorfismo. Al guardar objetos derivados en punteros de la clase base, el mismo llamado ejecuta la versión de la clase real del objeto. Esto se decide en tiempo de ejecución (**enlace dinámico**).

Destructor virtual. Necesario para que al hacer `delete` sobre un puntero de la clase base se llame también al destructor de la clase hija.

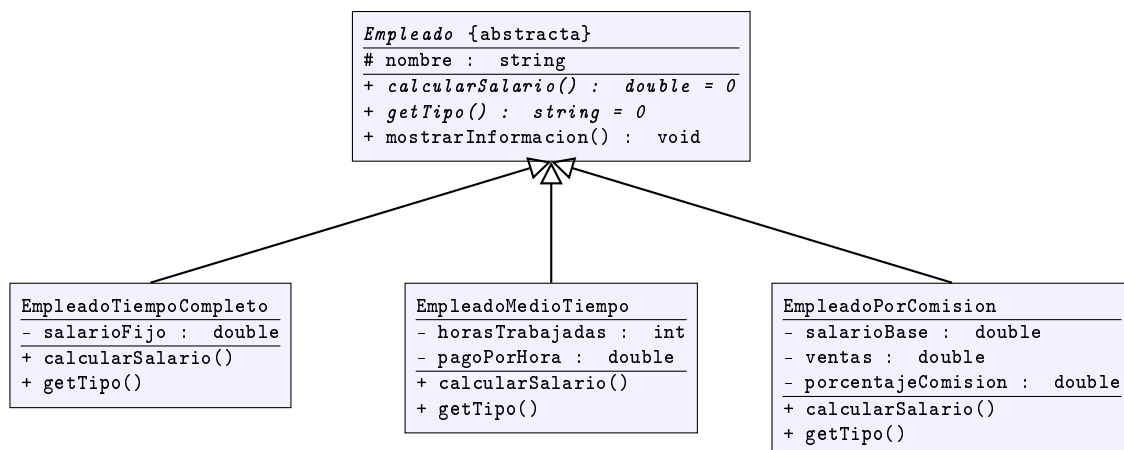
protected. El atributo común (**nombre o marca**) es `protected` para que las hijas puedan usarlo, pero sigue oculto desde fuera.

1 Ejercicio 01. (10 puntos) Sistema de empleados

Enunciado (resumen)

Crear la clase abstracta `Empleado` con el atributo `nombre`, el método virtual puro `calcularSalario()` y el método `mostrarInformacion()`. De ella derivan `EmpleadoTiempoCompleto`, `EmpleadoMedioTiempo` y `EmpleadoPorComision`, cada uno con su propia forma de calcular el salario. Luego se guardan en una colección de punteros a `Empleado` y se muestran usando polimorfismo.

Diagrama de clases



Cursiva: método abstracto; # protected, - private, + public.

Explicación de la solución

Clase Empleado. Es la clase abstracta. Tiene el atributo `nombre` y los métodos virtuales puros `calcularSalario()` y `getTipo()`, por eso no se puede instanciar. El método `mostrarInformacion()` está escrito una sola vez en la base, pero dentro llama a `calcularSalario()`, así que cada objeto muestra su propio salario.

Clases derivadas. Cada una llama al constructor del padre (`: Empleado(nombre)`) y calcula el salario de forma distinta:

- `EmpleadoTiempoCompleto`: Salario = `salarioFijo`
- `EmpleadoMedioTiempo`: Salario = `horasTrabajadas` × `pagoPorHora`
- `EmpleadoPorComision`: Salario = `salarioBase` + `ventas` × $\frac{\text{porcentaje}}{100}$

Función main(). Se crea un `vector<Empleado*>` con un empleado de cada tipo. Un solo bucle llama a `mostrarInformacion()` en todos sin preguntar de qué tipo son, eso lo resuelve el polimorfismo. Al final se libera la memoria con `delete`.

Código completo: ejercicio01_empleados.cpp

```
1 #include <iostream>
2 #include <iomanip>
3 #include <string>
4 #include <vector>
5 using namespace std;
6
7 class Empleado {
8 protected:
9     string nombre;
10
11 public:
12     Empleado(const string& nombre) : nombre(nombre) {}
13     virtual ~Empleado() {}
14
15     // virtual puro, cada tipo de empleado lo calcula a su manera
16     virtual double calcularSalario() const = 0;
17     virtual string getTipo() const = 0;
```

```

18
19     void mostrarInformacion() const {
20         cout << "Nombre : " << nombre << endl;
21         cout << "Tipo   : " << getTipo() << endl;
22         cout << fixed << setprecision(2);
23         cout << "Salario: S/ " << calcularSalario() << endl;
24     }
25 };
26
27 class EmpleadoTiempoCompleto : public Empleado {
28 private:
29     double salarioFijo;
30
31 public:
32     EmpleadoTiempoCompleto(const string& nombre, double salarioFijo)
33         : Empleado(nombre), salarioFijo(salarioFijo) {}
34
35     double calcularSalario() const override {
36         return salarioFijo;
37     }
38
39     string getTipo() const override {
40         return "Tiempo completo";
41     }
42 };
43
44 class EmpleadoMedioTiempo : public Empleado {
45 private:
46     int horasTrabajadas;
47     double pagoPorHora;
48
49 public:
50     EmpleadoMedioTiempo(const string& nombre, int horas, double pagoHora)
51         : Empleado(nombre), horasTrabajadas(horas), pagoPorHora(pagoHora) {}
52
53     double calcularSalario() const override {
54         return horasTrabajadas * pagoPorHora;
55     }
56
57     string getTipo() const override {
58         return "Medio tiempo";
59     }
60 };
61
62 class EmpleadoPorComision : public Empleado {
63 private:
64     double salarioBase;
65     double ventas;
66     double porcentajeComision; // 10 = 10%
67
68 public:
69     EmpleadoPorComision(const string& nombre, double base,
70                         double ventas, double porcentaje)
71         : Empleado(nombre), salarioBase(base),
72           ventas(ventas), porcentajeComision(porcentaje) {}

```

```

73
74     double calcularSalario() const override {
75         return salarioBase + ventas * (porcentajeComision / 100.0);
76     }
77
78     string getTipo() const override {
79         return "Por comision";
80     }
81 };
82
83 int main() {
84     vector<Empleado*> empleados;
85
86     empleados.push_back(new EmpleadoTiempoCompleto("Ana Torres", 3500.00));
87     empleados.push_back(new EmpleadoMedioTiempo("Luis Quispe", 80, 15.50));
88     empleados.push_back(new EmpleadoPorComision("Maria Flores", 1200.00, 20000.00, 10)
89 );
90
91     cout << "===== SISTEMA DE EMPLEADOS =====" << endl << endl;
92
93     double total = 0;
94     for (Empleado* e : empleados) {
95         e->mostrarInformacion(); // aqui se ve el polimorfismo
96         total += e->calcularSalario();
97         cout << "-----" << endl;
98     }
99     cout << "Total en planilla: S/ " << total << endl;
100
101     for (Empleado* e : empleados) {
102         delete e;
103     }
104     empleados.clear();
105
106     return 0;
107 }

```

Compilación y ejecución

```

g++ -std=c++17 -Wall -o ej1 ejercicio01_empleados.cpp
./ej1

```

Salida obtenida

```

===== SISTEMA DE EMPLEADOS =====

Nombre : Ana Torres
Tipo   : Tiempo completo
Salario: S/ 3500.00
-----
Nombre : Luis Quispe
Tipo   : Medio tiempo
Salario: S/ 1240.00
-----

```

```
Nombre : Maria Flores
```

```
Tipo   : Por comision
```

```
Salario: S/ 3200.00
```

```
-----
```

```
Total en planilla: S/ 7940.00
```

Comprobación

Medio tiempo: $80 \times 15.50 = 1240$. Por comisión: $1200 + 20000 \times 0.10 = 3200$. Total: $3500 + 1240 + 3200 = 7940$. Coincide con la salida del programa.

Conclusión

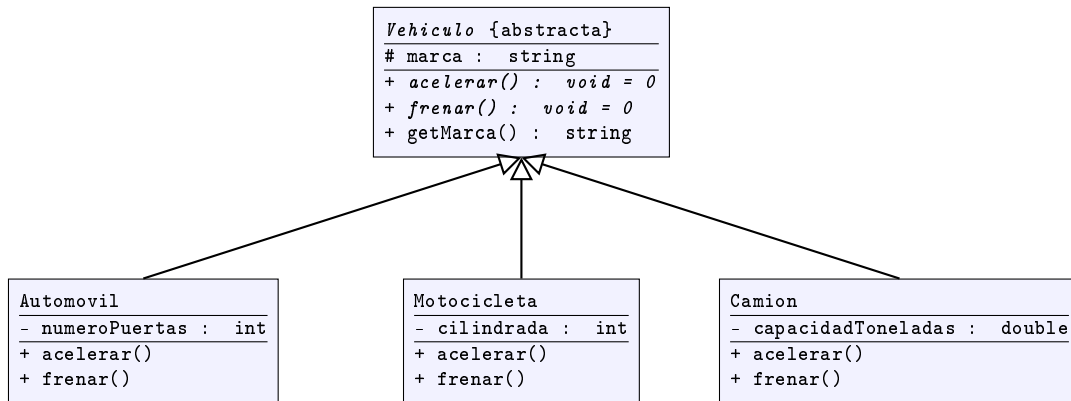
La clase abstracta **Empleado** define la interfaz común y cada hija pone su forma de calcular el salario. Para agregar otro tipo de empleado basta con crear una nueva clase, sin cambiar el bucle del **main**.

2 Ejercicio 02. (10 puntos) Sistema de vehículos

Enunciado (resumen)

Crear la clase abstracta `Vehiculo` con el atributo `marca` y los métodos virtuales puros `acelerar()` y `frenar()`. De ella derivan `Automovil`, `Motocicleta` y `Camion`, cada una con su propio comportamiento. Después se guardan en un `vector<Vehiculo*>` y se recorre el vector ejecutando ambos métodos con polimorfismo.

Diagrama de clases



Explicación de la solución

Clase Vehiculo. Clase abstracta con el atributo `marca`, destructor virtual y los métodos virtuales puros `acelerar()` y `frenar()`. Solo se usa como tipo de puntero.

Clases derivadas. Cada una tiene un atributo propio e implementa los dos métodos con `override`:

- **Automovil** (número de puertas): acelera suave y frena con disco y ABS.
- **Motocicleta** (cilindrada): acelera rápido y frena con ambos frenos.
- **Camion** (toneladas): acelera lento y frena con freno de aire.

Función main(). Se crea el `vector<Vehiculo*>` con los tres vehículos y se recorre llamando `v->acelerar()` y `v->frenar()`. Aunque el puntero es `Vehiculo*`, se ejecuta la versión de cada clase (enlace dinámico). Al final se libera la memoria.

Código completo: ejercicio02_vehiculos.cpp

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 using namespace std;
5
6 class Vehiculo {
7 protected:
8     string marca;
9
10 public:
11     Vehiculo(const string& marca) : marca(marca) {}
12     virtual ~Vehiculo() {}
13 }
```

```

14     virtual void acelerar() const = 0;
15     virtual void frenar() const = 0;
16
17     string getMarca() const { return marca; }
18 };
19
20 class Automovil : public Vehiculo {
21 private:
22     int numeroPuertas;
23
24 public:
25     Automovil(const string& marca, int puertas)
26         : Vehiculo(marca), numeroPuertas(puertas) {}
27
28     void acelerar() const override {
29         cout << "[Automovil " << marca << " - " << numeroPuertas
30             << " puertas] Pisa el acelerador y sube a 100 km/h de forma suave." <<
endl;
31     }
32
33     void frenar() const override {
34         cout << "[Automovil " << marca
35             << "] Frena con sus frenos de disco y ABS hasta detenerse." << endl;
36     }
37 };
38
39 class Motocicleta : public Vehiculo {
40 private:
41     int cilindrada; // cc
42
43 public:
44     Motocicleta(const string& marca, int cc)
45         : Vehiculo(marca), cilindrada(cc) {}
46
47     void acelerar() const override {
48         cout << "[Motocicleta " << marca << " - " << cilindrada
49             << " cc] Gira el punio y acelera rapido entre los autos." << endl;
50     }
51
52     void frenar() const override {
53         cout << "[Motocicleta " << marca
54             << "] Usa freno delantero y trasero a la vez para no derrapar." << endl;
55     }
56 };
57
58 class Camion : public Vehiculo {
59 private:
60     double capacidadToneladas;
61
62 public:
63     Camion(const string& marca, double capacidad)
64         : Vehiculo(marca), capacidadToneladas(capacidad) {}
65
66     void acelerar() const override {
67         cout << "[Camion " << marca << " - " << capacidadToneladas

```

```

68         << " t] Acelera lento por el peso de la carga." << endl;
69     }
70
71     void frenar() const override {
72         cout << "[Camion " << marca
73             << "] Activa el freno de aire y necesita mas distancia para parar." <<
endl;
74     }
75 };
76
77 int main() {
78     vector<Vehiculo*> vehiculos;
79
80     vehiculos.push_back(new Automovil("Toyota", 4));
81     vehiculos.push_back(new Motocicleta("Honda", 250));
82     vehiculos.push_back(new Camion("Volvo", 30));
83
84     cout << "==== SISTEMA DE VEHICULOS =====" << endl << endl;
85
86     // cada objeto ejecuta su propia version de acelerar y frenar
87     for (Vehiculo* v : vehiculos) {
88         v->acelerar();
89         v->frenar();
90         cout << "-----" << endl;
91     }
92
93     for (Vehiculo* v : vehiculos) {
94         delete v;
95     }
96     vehiculos.clear();
97
98     return 0;
99 }

```

Compilación y ejecución

```

g++ -std=c++17 -Wall -o ej2 ejercicio02_vehiculos.cpp
./ej2

```

Salida obtenida

```

===== SISTEMA DE VEHICULOS =====

[Automovil Toyota - 4 puertas] Pisa el acelerador y sube a 100 km/h de forma suave.
[Automovil Toyota] Frena con sus frenos de disco y ABS hasta detenerse.
-----
[Motocicleta Honda - 250 cc] Gira el punio y acelera rapido entre los autos.
[Motocicleta Honda] Usa freno delantero y trasero a la vez para no derrapar.
-----
[Camion Volvo - 30 t] Acelera lento por el peso de la carga.
[Camion Volvo] Activa el freno de aire y necesita mas distancia para parar.
-----

```

Conclusión

Con un solo bucle se hizo acelerar y frenar a tres vehículos distintos sin revisar su tipo. La clase abstracta asegura que todos tengan esos métodos y cada derivada decide cómo hacerlo.

Conclusiones generales

- Una clase con al menos un método virtual puro es abstracta y funciona como interfaz común.
- Las derivadas deben implementar esos métodos para poder crear objetos.
- Usar punteros de la clase base permite tratar a todos los objetos igual y que cada uno responda a su manera.
- El polimorfismo facilita agregar nuevas clases sin modificar el código que las usa.